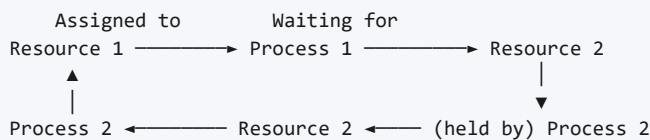


Deadlocks

1. What is a Deadlock?

Definition: A set of processes is in a deadlock if each process in the set is waiting for an event that only another process in the set can cause.

Classic everyday analogy: Amy holds the TV remote and wants the laptop; John holds the laptop and wants the TV remote. Neither will let go first — both wait forever.



Resource Allocation Graph convention:

- Arrow **from resource** → **process**: the resource is currently **held by** that process.
- Arrow **from process** → **resource**: the process is **requesting** that resource.

A **cycle** in this graph (with only one instance of each resource) is both necessary and — in that single-instance case — sufficient proof of deadlock.

Safe sequence: if there exists *some* order in which all processes can be run to completion without anyone getting permanently stuck, the system is *not* deadlocked (even if it currently looks contested).

2. Practical Reality: OS Design Philosophy Around Deadlocks

- There is **no fully efficient general solution** to deadlocks — every prevention/avoidance strategy costs something (throughput, resource utilization, complexity).
 - **Most general-purpose OSes (like Linux, Windows) mostly don't actively prevent or avoid deadlocks system-wide** — they let them happen and rely on other mechanisms (timeouts, manual intervention, killing processes) rather than paying the constant overhead of avoidance.
 - **Why:** the cost of deadlock (rare, and often resolvable by killing/restarting the stuck processes) is judged to be smaller than the cost of deadlock-avoidance machinery running on *every* resource request, all the time. This is a genuine engineering trade-off, not laziness.
-

3. Resource Types

- **Preemptable resource** — can be taken away from a process without ill effect (e.g., CPU, memory — a process can be swapped out and resumed later).
- **Non-preemptable resource** — cannot be taken away without causing failure/inconsistency (e.g., a printer mid-print, a file lock, a chopstick).

Deadlock is only possible with non-preemptable resources. If a resource could simply be forcibly taken back from a process, there would be no permanent "stuck waiting forever" — the OS could always break the standoff by preempting something.

4. The Four Necessary Conditions for Deadlock

All four must hold **simultaneously** for deadlock to be possible:

1. **Mutual exclusion** — at least one resource must be held in a non-shareable mode (only one process can use it at a time).
2. **Hold and wait** — a process holding at least one resource is waiting to acquire additional resources currently held by other processes.
3. **No preemption** — resources cannot be forcibly taken from a process; they can only be released voluntarily.
4. **Circular wait** — there exists a cycle of processes, each waiting for a resource held by the next one in the cycle.

Necessary vs sufficient: all four are *necessary* — remove any single one, and deadlock becomes impossible. But **circular wait is also sufficient** (given only one instance of each resource in the cycle) — if a circular wait exists under single-instance resources, deadlock has already occurred; you don't need to separately re-verify the other three.

Interview soundbite: "Deadlock requires all four conditions — mutual exclusion, hold-and-wait, no preemption, and circular wait — to hold at once. Break any one of them, and deadlock becomes structurally impossible. Prevention strategies each target exactly one of these four."

5. Three Strategies to Handle Deadlocks

Strategy	When it acts	Idea
Prevention	At design time	Structure the OS/system so at least one of the four conditions can <i>never</i> hold
Avoidance	At resource-allocation time	Allow all four conditions to be possible, but carefully decide <i>whether to grant</i> each request so the system never actually enters an unsafe state
Recovery	After deadlock has occurred	Detect it, then break it (kill a process, preempt a resource, roll back)

6. Deadlock Prevention — Disallowing One Condition

6.1 Disallow Mutual Exclusion

Make all resources shareable. **Not practical** — resources like printers, chopsticks, or write-locks on a file are inherently exclusive by nature; you can't safely pretend otherwise.

6.2 Disallow Hold and Wait

Force a process to request **all** the resources it will ever need **up front**, before starting — or force it to release everything it holds before requesting anything new.

- **Consequence:** poor resource utilization (a process holds resources it isn't using yet) and potential **starvation** (a process needing many resources may wait a long time for all of them to be simultaneously available).

6.3 Disallow No Preemption

Allow the OS to forcibly take a resource away from a process (and roll that process back to an earlier state).

- **Consequence:** not all resources *can* be safely preempted (you can't "un-print" half a page or safely interrupt a non-idempotent operation). Where possible, it also causes **computational loss** — the preempted process's progress since it acquired that resource may need to be redone (rollback).

6.4 Disallow Circular Wait

Order all resources (assign each a unique ID/number), and require every process to request resources **only in strictly increasing order** of that ID.

- **Why this works:** if every process must request R1 before R2 before R3, no cycle can ever form — a cycle would require some process to go "backward" in the ordering to complete the loop, which the rule forbids.
- **Consequence:** can lead to **poor resource utilization** (a process might be forced to acquire a low-numbered resource it doesn't need yet, just to be allowed to later request a high-numbered one it actually needs) and **longer waiting times**.

Common theme across all four: each prevention technique *works*, but each one sacrifices something — utilization, throughput, or flexibility. This is why prevention isn't free, and why real systems mostly don't bother enforcing it globally (see §2).

7. Deadlock Avoidance — Banker's Algorithm

Avoidance doesn't disallow any of the four conditions — instead, it carefully evaluates **every resource request** and only grants it if doing so keeps the system in a **safe state**.

Key terms:

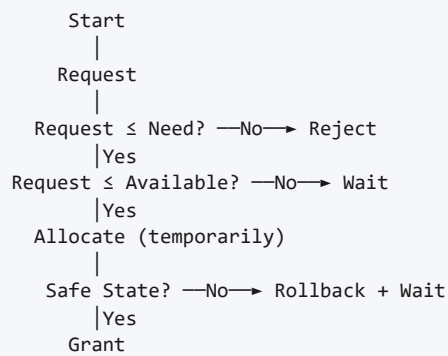
- **Available** — resources currently free.
- **Allocation** — resources currently held by each process.
- **Max** — the maximum a process might ever need.
- **Need = Max – Allocation** — what a process might still request.

Safe state: a state from which there exists **some** order to run all processes to completion (each getting its full **Need** satisfied in turn, then releasing its resources back to **Available** for the next). If no such order exists, the state is **unsafe** — not necessarily already deadlocked, but one bad request away from it.

Analogy (from your notes): a bank cashier with a fixed cash reserve, facing a queue of customers who might each eventually ask for up to their stated maximum withdrawal. The cashier only approves a withdrawal if, after granting it, there's *still* some guaranteed way to satisfy everyone's maximum eventual demand using remaining cash plus expected repayments — even if that means temporarily saying no to a request that could technically be covered right now.

Algorithm (per request):

```
If (Request ≤ Need AND Request ≤ Available):  
    Pretend to allocate (temporarily)  
    If the resulting state is SAFE → actually Grant  
    Else → Rollback (pretend-undo) and make the process Wait  
Else:  
    Wait (request itself is invalid or resources aren't currently free)
```



Why this prevents deadlock: by only ever granting requests that keep the system in a state with *some* guaranteed safe completion order, the algorithm ensures the system can never slide into an actual deadlock — every reachable state has an escape route.

Cost: requires knowing each process's maximum future resource need **in advance** (often unrealistic) and re-running the safety check on every single request (computational overhead).

8. Deadlock Recovery

Once a deadlock is **detected** (via periodic cycle-detection over the resource allocation graph, or similar), recovery options include:

- **Process termination**
- Kill *all* deadlocked processes (blunt, but simple).
- Kill them **one at a time**, re-checking for deadlock after each, until it's resolved (less wasteful, more overhead per step).
- **Resource preemption**
- Forcibly take a resource from one of the deadlocked processes and give it to another, breaking the cycle.
- Requires rolling that process back to a safe earlier state (since it lost a resource it was relying on) — same computational-loss trade-off as in prevention.

Choosing a victim for termination/preemption typically considers: process priority, how long it's run / how much it's completed, how many resources it holds, how many more it needs, and how many other processes would need to be rolled back as a consequence.

Interview Questions With Answers

Q1. Define deadlock formally.

Answer: A set of processes is deadlocked if every process in the set is waiting for an event (typically resource release) that can only be caused by another process in that same set — meaning none of

them can ever proceed.

Q2. What are the four necessary conditions for deadlock, and why must all four hold simultaneously?

Answer: Mutual exclusion, hold-and-wait, no preemption, and circular wait. All four must hold simultaneously because breaking even one makes deadlock structurally impossible: e.g., if preemption were allowed, a resource could just be forcibly reclaimed to break any standoff, so "no preemption" failing to hold removes the deadlock risk entirely, regardless of the other three.

Q3. Is circular wait a necessary condition, a sufficient condition, or both?

Answer: Both, in the single-instance-per-resource-type case. It's necessary because deadlock can't occur without some cycle of waiting. It's also sufficient in that specific case — if you can show a cycle exists in the resource allocation graph where each resource has only one instance, that alone proves deadlock, without needing to separately check the other three conditions (they're implied).

Q4. Why can deadlock only happen with non-preemptable resources?

Answer: If a resource could be forcibly taken from a process without ill effect, the OS could always resolve any standoff by preempting a resource from one of the waiting processes and handing it to another — breaking the cycle. Deadlock specifically requires that resources can only be released voluntarily, which is exactly the definition of non-preemptable.

Q5. Why don't most general-purpose operating systems actively prevent or avoid deadlocks?

Answer: Because prevention and avoidance both carry constant, real costs — reduced resource utilization, added waiting time, or per-request computational overhead (as in Banker's algorithm) — paid on every single resource request, all the time. Deadlocks in practice are relatively rare and are typically resolvable after the fact by killing/restarting the stuck process(es). OS designers judge that ongoing avoidance overhead usually costs more than occasionally recovering from a rare deadlock.

Q6. How does disallowing "hold and wait" prevent deadlock, and what's the cost?

Answer: By forcing a process to request all resources it will ever need up front (or release everything before requesting more), no process can ever be caught holding one resource while waiting on another — which is exactly what hold-and-wait requires. The cost is poor resource utilization (holding resources before you actually need them) and potential starvation (a process needing many resources might rarely find all of them free simultaneously).

Q7. Explain how ordering resources prevents circular wait.

Answer: If every resource is assigned a unique ID and every process is required to request resources only in strictly increasing order of that ID, no cycle can ever form — completing a cycle would require

at least one process to request a lower-numbered resource after already holding a higher-numbered one, which the ordering rule explicitly forbids.

Q8. What is a "safe state" in the context of Banker's algorithm? Is an unsafe state the same as a deadlock?

Answer: A safe state is one where there exists at least one ordering in which every process can eventually get its full maximum need satisfied and complete, given the currently available resources plus what's freed as each process finishes. An unsafe state is **not** the same as deadlock — it just means no such guaranteed-safe ordering currently exists; the system *might* still avoid deadlock depending on what happens next, but it's no longer guaranteed to.

Q9. What information does Banker's algorithm require that makes it hard to use in practice?

Answer: It requires knowing each process's **maximum** possible future resource need in advance, which is often unrealistic for general-purpose systems where processes don't declare their full resource appetite ahead of time. It also requires re-running the safety check on every single resource request, which adds ongoing computational overhead.

Q10. What are the two main categories of deadlock recovery, and what's the shared cost between them?

Answer: Process termination (killing all deadlocked processes at once, or one at a time with re-checking) and resource preemption (forcibly taking a resource from one deadlocked process to give to another). Both share the cost of **lost work**: termination throws away a process's progress, and preemption typically requires rolling the affected process back to an earlier safe state, redoing work it had already done.

Q11. Scenario: Process P1 holds resource R1 and requests R2. Process P2 holds R2 and requests R1. Both resources have exactly one instance. Is this a deadlock? How do you know without simulating further?

Answer: Yes — this is a two-process circular wait ($P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$), and since each resource has only one instance, circular wait is not just necessary but *sufficient* to prove deadlock here. No further simulation is needed: this cycle alone guarantees neither process can ever proceed, since each is waiting on a resource permanently held by the other.

Q12. Scenario: If R1 in the above question had 2 available instances instead of 1, would it still necessarily be a deadlock?

Answer: Not necessarily. With multiple instances of a resource type, a cycle in the resource allocation graph is still *necessary* for deadlock but no longer automatically *sufficient* — it's possible that another process could release an instance of R1 that satisfies P2's request without needing P1 to release anything first, breaking the cycle without anyone being permanently stuck. You'd need to

actually check whether some safe completion order exists (essentially applying the same reasoning as Banker's algorithm), rather than concluding deadlock from the cycle alone.